Expert Systems With Applications 269 (2025) 126479

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

SmartGuard: An LLM-enhanced framework for smart contract vulnerability detection

Hao Ding [a], Yizhou Liu [a], Xuefeng Piao [a,*], Huihui Song [b], Zhenzhou Ji [a]

[a] School of Computer Science and Engineering, Harbin Institute of Technology, Harbin, 150000, Heilongjiang, China
[b] School of New Energy, Harbin Institute of Technology, Weihai, 264209, Shandong, China

ARTICLE INFO

Keywords:
Vulnerability detection
In-context learning
Chain of thought
Smart contract
Blockchain

ABSTRACT

Blockchain technology has seen widespread adoption across various sectors, making the security of smart contracts essential for the reliable operation of blockchain networks. Conventional static detection tools often fall short, suffering from false negatives due to limited semantic analysis and dependence on predefined rules. In contrast, deep learning-based methods offer superior performance by continuously updating their knowledge base and learning the nuanced characteristics of potential vulnerabilities. This paper presents SmartGuard, an enhanced framework for smart contract vulnerability detection that leverages Large Language Models (LLMs). SmartGuard identifies vulnerabilities by retrieving semantically similar code snippets, generating corresponding Chains of Thought (CoT), and combining these elements for in-context learning. To optimize the use of labeled data for automatic CoT generation, we introduce a self-check architecture that utilizes the self-correcting capabilities of LLMs to produce high-quality reasoning chains. Evaluations on the SolidiFI benchmark dataset demonstrate a recall of 95.06% and an F1-score of 94.95%, outperforming existing tools. Further experiments across multiple models confirm the framework's generalizability. The code is publicly available at https://github.com/KevinClaint/SmartGuard.

1. Introduction

Since the introduction of Bitcoin in 2009, blockchain, as a revolutionary technology, has garnered significant attention and has been widely referenced across various fields (Li, Jiang, Chen, Luo, & Wen, 2020; Xu et al., 2024). It serves as a distributed database technology ensuring data immutability and transparency through encryption and consensus mechanisms, commonly applied in cryptocurrencies and supply chain management.

Smart contracts are small programs deployed on the blockchain that automatically enforce terms and conditions between two untrusted parties (Clack, Bakshi, & Braine, 2016). These contracts manage funds or data on the blockchain using predefined business logic (Alharby & Van Moorsel, 2017). However, their flexibility introduces substantial security risks. In a secure blockchain environment, smart contracts are highly dependent on their code logic, and any flaws in the implementation can severely compromise blockchain security (Lu, 2019). These logical flaws are known as vulnerabilities. A timestamp vulnerability, for instance, occurs when a smart contract relies on a blockchain timestamp to execute critical operations, allowing a malicious user to manipulate the timestamp to alter the contract's behavior during execution. Numerous security incidents have arisen due to such vulnerabilities. For example, in 2016, hackers exploited a reentrancy vulnerability in the DAO contract, stealing approximately $60 million worth of Ether from the asset pool. Similarly, in 2018, an integer overflow vulnerability in the BEC contract led to the token's value dropping to zero. In 2024, an accuracy calculation vulnerability in the MIM_SPELL smart contract was exploited, resulting in a loss of about $6.5 million. These incidents highlight the severe financial losses that vulnerabilities in smart contracts can cause (Sayeed, Marco-Gisbert, & Caira, 2020). Additionally, unlike conventional software, smart contracts are immutable, making it difficult to fix vulnerabilities after deployment on the blockchain. Therefore, conducting thorough security testing before deployment is essential.

Presently, smart contract vulnerability detection methods mainly include formal verification (Bai, Cheng, Duan, & Hu, 2018; Park, Zhang, Saxena, Daian, & Roşu, 2018; Sun & Yu, 2020), static analysis (Brent et al., 2018; Feist, Grieco, & Groce, 2019; Kalra, Goel, Dhawan, & Sharma, 2018; Tikhomirov, Voskresenskaya, Ivanitskiy, Takhaviev, Marchenko, & Alexandrov, 2018; Tsankov, Dan, Drachsler-Cohen, Gervais, Buenzli, & Vechev, 2018), symbolic execution (E., 2019; Luu,

* Corresponding author.
E-mail addresses: dinghao@stu.hit.edu.cn (H. Ding), 2021211107@stu.edu.cn (Y. Liu), hbpark@hit.edu.cn (X. Piao), songhh@hitwh.edu.cn (H. Song), jizhenzhou@hit.edu.cn (Z. Ji).

https://doi.org/10.1016/j.eswa.2025.126479
Received 12 October 2024; Received in revised form 2 December 2024; Accepted 7 January 2025
Available online 13 January 2025
0957-4174/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Chu, Olickel, Saxena, & Hobor, 2016; Mossberg et al., 2019), and fuzzy detection (Jiang, Liu, & Chan, 2018). However, these methods heavily rely on predefined detection rules and often make erroneous judgments for codes with complex logic. With the rapid development of smart contracts, it is foreseeable that vulnerabilities beyond predefined detection rules will emerge. In contrast, deep learning methods learn logical features in the code, providing a deeper understanding. They also offer the potential for further learning through data updates.

In the past few years, deep learning-based smart contract vulnerability detection methods have been emerging (Gogineni, Swayamjyoti, Sahoo, Sahu, & Kishore, 2020; Huang, Zhou, Xiong, & Li, 2022; Lutz et al., 2021; Tann, Han, Gupta, & Ong, 2018). However, these methods typically require large volumes of carefully labeled datasets for training. Additionally, and most importantly, current deep learning approaches (Gogineni et al., 2020; Gupta, Bansal, Sharma, Mehrotra, & Kakkar, 2024; Huang et al., 2022; Lutz et al., 2021; Tann et al., 2018) are mainly based on discriminative models (*e.g.*, CNN, LSTM, RNN, GNN, etc.) to determine the type of vulnerability through probability. In scenarios where multiple vulnerabilities exist within smart contracts, common in practice, these models struggle to detect multiple issues in a single pass. Consequently, this can result in errors due to the influence of multiple vulnerabilities on the detection process. In contrast, Large Language Models (LLMs) typically utilize generative models that can directly convey information, such as the type of vulnerability, through natural language output, thereby avoiding this issue.

In recent years, LLMs (Chowdhery et al., 2023; Du et al., 2022; Thoppilan et al., 2022; Touvron et al., 2023; Zeng et al., 2022) have achieved significant success. They have demonstrated remarkable capabilities in complex reasoning tasks when given appropriate prompts (Wei et al., 2022). There is also a growing exploration of the application of LLMs in the context of blockchain security (Boi, Esposito, & Lee, 2024). Chen et al. Chen et al. (2023) propose the use of LLMs to detect vulnerabilities in smart contracts, which demonstrates the potential of LLMs in this regard, while they also point out that the accuracy of LLMs in pinpointing smart contract vulnerabilities is limited. It is worth noting that the introduction of Chain of Thought (CoT) prompts (Wei et al., 2022) has greatly improved the reasoning of linguists, including arithmetic, commonsense, and symbolic reasoning. Sun et al. Sun et al. (2024) introduced an external knowledge base and allowed LLMs to use the CoT approach to reasoning with good results. They emphasized that enhanced knowledge is crucial to help LLMs better understand code and vulnerabilities. However, most of them have not considered or attempted to drive large models to address the problem of multiple vulnerabilities in a single code snippet, or to propose a workable solution.

In this paper, we introduce an LLM-enhanced framework for smart contract vulnerability detection named SmartGuard. The framework comprises four steps for determining smart contract vulnerabilities: (1) Code Selection: Select code snippets from the labeled dataset with high semantic similarity to the test code snippet. (2) CoT Generation: LLMs generate CoT sequences for selected code snippets, ensuring consistency with their labels through Self-Check. (3) In-Context Learning: Combine the selected code snippets with their corresponding CoT sequences, embedding them in the prompt. (4) Answer Inference: Enable LLMs to reason about the test code based on the provided prompt. The main contributions of this paper are as follows:

- We introduce SmartGuard, a framework that utilizes LLMs to enhance the detection of vulnerabilities in smart contracts by selecting CoT prompts. The proposed SmartGuard can autonomously generate CoTs from code corpus snippets and perform smart contract vulnerability detection.
- We propose a novel approach that optimizes the use of labeled datasets for the automatic generation of high-quality rational chains, eliminating the need for costly manual labeling of datasets that lack such rational chains.
- We provide a comprehensive comparison between the SmartGuard approach and common smart contract vulnerability detection tools and demonstrate the advantages in terms of performance and generalizability. Through experiments on multiple LLMs, we demonstrate the effectiveness of the framework, including high recall and F1 scores on the SolidiFI benchmark dataset, outperforming current tools.

The rest of this paper is organized as follows. The related work is surveyed in Section 2. Section 3 provides an overview of our framework. In Section 4, indicators for evaluating our framework are presented. Subsequently, the experiment details are provided in Section 5. Section 6 then proceeds to validate and analyze the proposed strategy, followed by a discussion of our work in Section 7. Additionally, this paper concludes with a summary in Section 8.

2. Related work

2.1. Static analysis techniques

Static analysis refers to the process of analyzing and inspecting code without executing the program, aiming to identify and fix vulnerabilities present in the code. In the realm of smart contract vulnerability detection, many static analysis tools already exist. Here, we briefly outline several prominent ones:

Securify (Tsankov et al., 2018) is a smart contract formal verification tool grounded in facts and rules. It operates by transforming EVM bytecode into a static single assignment form and subsequently analyzes it to deduce semantic facts. These inferred facts are then cross-referenced against predefined security property rules to detect vulnerabilities. However, due to its heavy reliance on predefined rules, Securify may overlook or produce erroneous results when vulnerabilities lie beyond the scope of these rules.

Smartcheck (Tikhomirov et al., 2018) identifies vulnerabilities through lexical and syntactic analysis of Solidity source code. It leverages the lexical analysis tool ANTLR alongside custom Solidity syntax to generate an XML parse tree as an intermediate representation. XPath queries are then applied to this representation to match security properties. Despite its capabilities, Smartcheck may require additional assistance to address complex vulnerabilities such as taint analysis and manual review. Moreover, its lack of deep code understanding makes it susceptible to false positives or negatives.

Slither (Feist et al., 2019) conducts vulnerability detection and code comprehension through the analysis of intermediate representations known as SlithIR. SlithIR is constructed from an abstract syntax tree of smart contract code, and contract information is recovered through contract inheritance graphs, control flow graphs, etc. In comparison to Securify and Smartcheck, Slither demonstrates substantial enhancements in accuracy and recognition rates. However, as Slither lacks a formal semantic analysis process, it is restricted to analyzing solely the surface structure of the code, without delving into its underlying meaning and logic. Consequently, this limitation impedes Slither's capability to detect certain intricate issues, including specific security concerns arising from logic errors or business logic within the code.

2.2. Large language models

LLMs represent a significant breakthrough in artificial intelligence, specifically in the field of natural language processing (Hou et al., 2023). These models are typically pre-trained on extensive datasets and are not fine-tuned for specific tasks. Their substantial size, depth, and complexity enable them to process textual input and generate fluent textual responses that match human cognition (Yang et al., 2024).

The primary distinction between LLMs and traditional language models lies in their enormous scale. This is generated by training with text data containing billions or even trillions of words. This task-unspecific training data allows LLMs to capture the complex nuances of

language, including syntax, semantics, and pragmatics, thus granting them an in-depth understanding of language structure and conventions (Kang, Yoon, & Yoo, 2023). However, their massive size often necessitates advanced hardware for deployment, posing a challenge for some traditional computer practitioners. Consequently, many users opt to access LLMs through API calls to overcome these hardware limitations.

2.3. In-context learning

Recent studies have demonstrated that as LLMs scale up in size (Brown et al., 2020; Radford et al., 2019), LLMs exhibit robust in-context learning (ICL) capabilities, particularly evident in tackling complex tasks such as solving mathematical reasoning problems (Wei, Wang et al., 2022). ICL represents a paradigm wherein LLMs acquire proficiency in a designated task when provided with several given examples (Dong et al., 2022). This approach empowers LLMs to address diverse problems without necessitating gradient updates, thereby substantially mitigating the extensive time and financial resources typically associated with model training.

In ICL, the choice of demonstrations in the prompts and the sequence in which these demonstrations are presented can significantly impact the effectiveness demonstrated by the models (Zhao, Wallace, Feng, Klein, & Singh, 2021). Regarding demonstration selection, Levy, Bogin, and Berant (2022) advocate for employing diverse demonstrations to enhance the model's generalization performance. They promote the idea of selecting demonstrations from different distributions to encourage the model to generate responses beyond the demonstrated examples. Additionally, Liu et al. Liu, Shen, Zhang, Dolan, Carin, and Chen (2021) propose a kNN-based unsupervised retriever for context-based demonstration selection. They establish that selecting semantically similar demonstrations enhances the model's performance in ICL. The semantic similarity of sentences is typically assessed using metrics such as the L2 distance or cosine similarity distance of sentence embeddings. Concerning the order of demonstrations, Lu, Bartolo, Moore, Riedel, and Stenetorp (2022) prove that in ICL, models are sensitive to the arrangement of demonstrations in the prompts. They suggest using their definitions of Global Entropy and Local Entropy to determine the optimal example ordering, where the entropy measure correlates positively with the model's performance in ICL. Furthermore, Liu et al. (2021) argue that arranging the most similar sentences near the test example may enhance the model's performance in ICL.

2.4. Chain-of-thought prompt

Chain of Thought (CoT) is a prompting paradigm introduced by Wei, Wang et al. (2022). It involves generating representations by incorporating intermediate reasoning steps between the input and output. These steps may include listing hypotheses, reasoning, applying formulas, or providing context-based explanations. CoT aids models in understanding complex problems by explicitly illustrating the reasoning process. As a result, LLMs can enhance their proficiency in complex reasoning tasks by mastering the various intermediate steps in CoT prompts (Kojima, Gu, Reid, Matsuo, & Iwasawa, 2022), thereby progressively processing information to achieve more accurate and reliable results.

CoT generation is currently characterized by manual annotation and automatic annotation approaches. Wei, Wang et al. (2022) employ effective manual demonstration to elicit CoT reasoning ability, a method referred to as Manual-CoT. However, the substantial human effort required to design the problem and its chain of reasoning significantly increases costs. Kojima et al. (2022) enhance LLMs' reasoning ability by employing "Let's think step by step" prompts to guide them through problem-solving incrementally, also known as zero-shot learning. Zhang, Zhang, Li, and Smola (2022) propose Auto-CoT, enabling LLMs to generate chains of reasoning for demonstrations through "Let's think step by step" prompts. To minimize errors in the inference chain, Auto-CoT selects representative problems from different categories for sampling and generates reasoning chains to construct the demonstration. To maximize the utility of labeled data, Shum, Diao, and Zhang (2023) introduce Automatic-CoT, which automatically generates rational reasoning chains and prunes low-quality reasoning chains based on labels. Additionally, they employ a variance-reduced policy gradient strategy to select the optimal CoT.

3. Design of SmartGuard

The overall framework of our proposed approach is depicted in Fig. 1. As illustrated in the figure below, our SmartGuard comprises four primary phases, each with its designated role:

- **Demonstration Selection Stage:** SmartGuard retrieves the top-k most similar code snippets to the test code from the smart contract corpus through semantic analysis.
- **Demonstration Expansion Stage:** SmartGuard utilizes LLMs to generate Code CoTs related to smart contract vulnerability detection for each of the top-k code snippets retrieved.
- **In-Context Learning Stage:** SmartGuard combines the retrieved top-k code snippets with their generated corresponding Code CoTs as a demonstration, thereby enabling LLMs to uncover potential relationships between the code and smart contract vulnerabilities. Additionally, this approach facilitates the exploration of the models' potential knowledge in relevant domains through ICL.
- **Inference Stage:** SmartGuard combines the base prompt with examples, inputs LLMs, and generates detection reports.

We will delve into each stage in greater detail in the subsequent sections.

3.1. Demonstration selection stage

In practice, smart contracts often include annotations to facilitate readability and modification by programmers. Accurate annotations may enable the model to bypass a comprehensive understanding of the code's logic, while inaccurate annotations could lead to misunderstandings by the model. To ensure the model grasps the logic and semantics of the code as accurately as possible, we construct the smart contract corpus with all comments removed from the code. Furthermore, to improve the quality of the inference chain generated in Section 3.2, we partition the smart contract into code snippets based on functions.

Zhao et al. (2021) have shown that the selection of demonstrations significantly impacts the inference performance of LLMs. Dong et al. (2022) further suggest that high-quality demonstrations, closely aligned with the objectives, can enhance LLMs' understanding of the tasks at hand. To select demonstrations that optimize modeling capabilities, we adopt a demonstration selection strategy similar to that of Liu et al. (2021). The process for selecting specific demonstrations is illustrated in Fig. 2.

In contrast to Liu et al. (2021), who use a pre-trained RoBERTa model (Liu et al., 2019) as a sentence encoder, we employ CodeBERT (Feng et al., 2020), which achieves state-of-the-art performance in programming language processing tasks. The semantic vectors generated by CodeBERT effectively capture the semantics of code snippets with high fidelity.

We first transform the selected code snippets $C_i = \{c_1, c_2, \ldots, c_n\}$ into sequences $E_i = \{e_1, e_2, \ldots e_n\}$ compatible with the model using the CodeBERT tokenizer, where n represents the number of words segmented out of the code snippet based on the word list, c_i represents the code snippet segmented out of the word based on the word list, and e_i represents the vector representation of c_i. This tokenizer employs an embedding method to convert text or symbolic data into vector

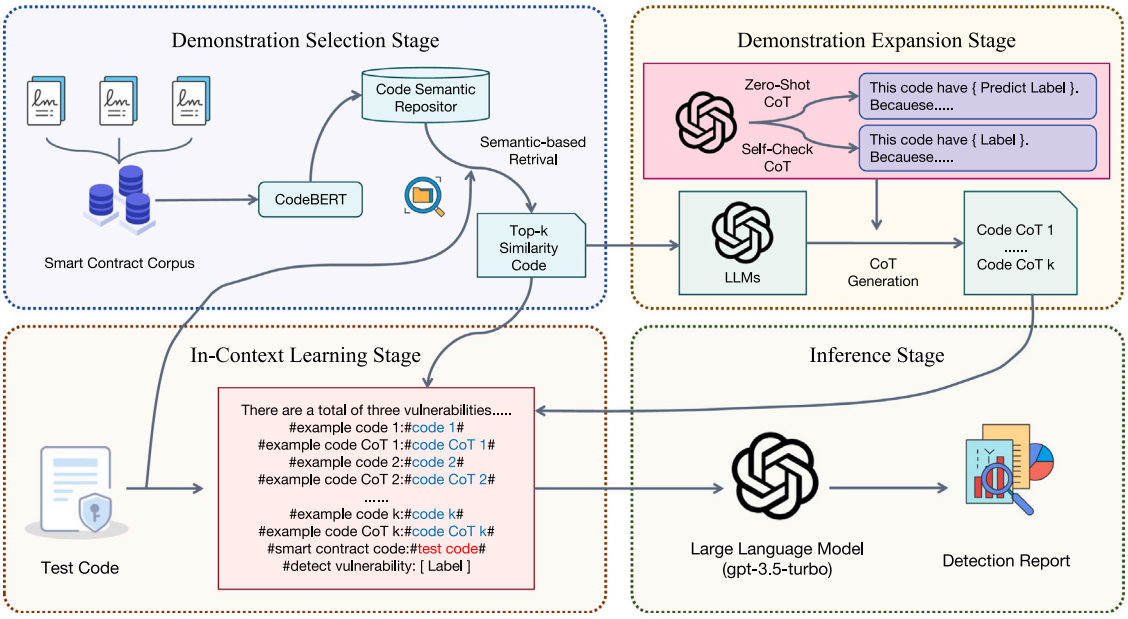


Fig. 1. Framework of our proposed approach SmartGuard.

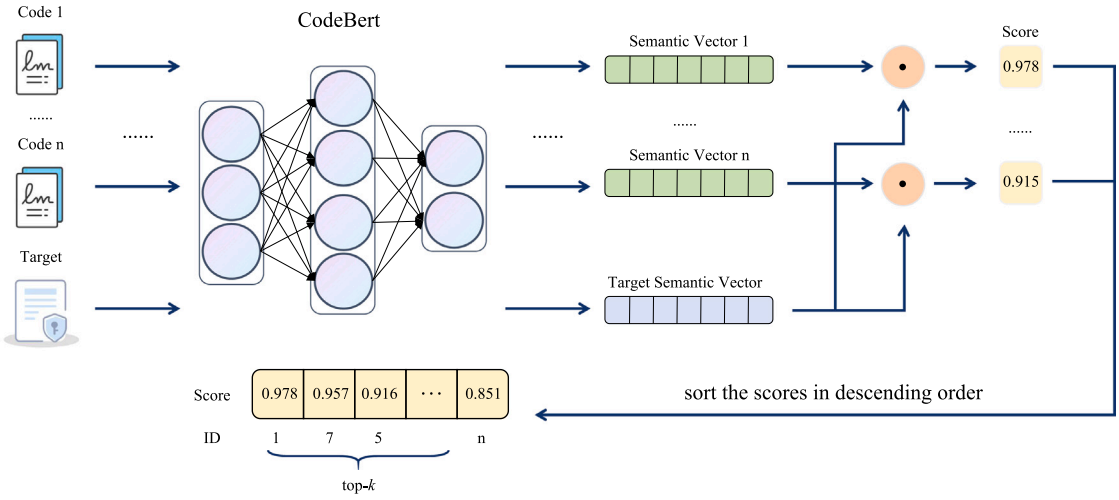


Fig. 2. The specific of demonstration selection.

representations, where each word is encoded as a 512-dimensional embedding.

Additionally, CodeBERT incorporates position embeddings, which enable the model to capture positional information within the sequence. These embeddings assign a specific vector to each position, reflecting the relative location of a token in the sequence. For a position i and dimension k, the position encoding PE(i,k) is computed using a predefined formula (1). The vector representation of the code snippet is then fed into CodeBERT, and the output from its final hidden layer serves as the semantic vector $X_i \in \mathbb{R}^N$, where N represents the dimension of the encoder's hidden layer.

$$PE(i,k) = \begin{cases} \sin\left(\frac{i}{10000^{k/d}}\right) & \text{if } k \text{ is even,} \\ \cos\left(\frac{i}{10000^{k/d}}\right) & \text{if } k \text{ is odd,} \end{cases} \tag{1}$$

Subsequently, the test code undergoes transformation into a semantic vector Y using the same approach. Then, we compute the cosine similarity between the semantic vector of the test code and the semantic vector of each code snippet in the smart contract code base. The calculation formula is as follows:

$$\text{Cosine Similarity}(\mathbf{X},\mathbf{Y}) = \frac{\sum_{i=1}^{n} x_i y_i}{\sqrt{\sum_{i=1}^{n} x_i^2}\sqrt{\sum_{i=1}^{n} y_i^2}} \tag{2}$$

Based on the results obtained from the calculation, the code snippets in the smart contract corpus are sorted according to their distance from the test code. A higher cosine similarity value indicates greater semantic similarity between the two. Subsequently, the top-k code snippets are selected from the sorted list to form part of the demonstration.

3.2. Demonstration expansion stage

CoT has been shown to enhance the reasoning capability of LLMs, enabling them to excel in complex reasoning tasks (Wei, Wang et al., 2022). However, many studies still rely on costly manually annotated reasoning chains, even when datasets with high-quality labels are available. Therefore, the effective utilization of well-labeled data to enhance the automatic generation of reasoning chains remains a significant concern. Shum et al. (2023) propose a method to automatically prune

low-quality reasoning chains generated by using labels, thus obtaining a low-cost corpus of high-quality inference chains. Their Automatic-CoT leverages data labeling to achieve better results. However, in cases where the dataset size is small, the number of available demonstrations in the high-quality reasoning chain repository may become even scarcer after pruning. While fewer or irrelevant demonstrations may still contribute to improving the model's reasoning capability to some extent, the model's performance can be further enhanced by selecting more demonstrations or choosing those that are more suitable for the problem at hand.

Inspired by the above work, and with the goal of generating as many high-quality reasoning chains as possible, we introduce a straightforward reasoning architecture called self-check, as outlined in the pseudo-code of Algorithm 1. This architecture aims to let LLMs self-check for error correction when they generate erroneous results. When the number of checks exceeds some upper limit, let LLMs generate reasoning chains based on the answers in reverse. In the remaining part of this subsection, we will show the details of this section.

Algorithm 1: Self-check of CoT with SmartGuard

Input : $N_t \leftarrow$ Rethinking Limit
Input : $N_c \leftarrow$ Self-Check Limit
Input : $C \leftarrow$ List of code snippets
Input : $L \leftarrow$ List of labels corresponding to code snippets
Input : $P \leftarrow$ Prompt for LLM
Input : $S \leftarrow$ SmartGuard
Output: Recorded CoTs

$k \leftarrow 0$; /* Snippets index */
$c \leftarrow C[k]$; /* First code snippet */
$l \leftarrow L[k]$; /* First label */
while $k < len(C)$ **do**
  $t \leftarrow 0$; /* Times of checking */
  for $t \leq N_t$ **do**
    $O \leftarrow S(P, c)$; /* Output of SmartGuard */
    if *The labels included in O are consistent with l* **then**
      Record the CoT contained in O;
      break;
    end
    else
      $h \leftarrow$ hints that an error has occurred;
      $P_t \leftarrow P + O + h$;
      $O \leftarrow S(P_t, c)$;
      $t \leftarrow t + 1$;
    end
  end
  if *t is equal to limit* N_t **then**
    $t_c \leftarrow 0$;
    for $t_c \leq N_c$ **do**
      $P_t \leftarrow P + l$; /* Embed the label l in prompt P */
      $O \leftarrow S(P_t, c)$;
      if *The labels included in O are consistent with l* **then**
        Record the CoT contained in O;
        break;
      end
    end
  end
  $t \leftarrow 0$;
  $k \leftarrow k + 1$;
  $c \leftarrow C[k]$;
  $l \leftarrow L[k]$;
end

Recently, several studies have shown that LLMs possess the capability to detect errors in their own output autonomously, indicating their ability for self-correction (Kim, Baldi, & McAleer, 2024; Saunders et al., 2022). Motivated by the above work, and aiming to generate as many high-quality reasoning chains as possible, we introduce a straightforward reasoning architecture called self-check. This architecture is designed to enable LLMs to self-check for error correction when they produce erroneous results. If the number of checks exceeds a predefined upper limit, the LLMs are tasked with generating reasoning chains based on the answers in reverse. In the subsequent part of this subsection, we will elucidate the details of this approach.

The specific workflow of self-check is illustrated in Fig. 3. The processed code snippet is combined with the underlying prompt. By employing zero-shot learning, which involves appending "Let's think step by step" at the end of the question, LLMs generate a CoT for detecting vulnerabilities in that code snippet. Subsequently, the labels of the smart contract vulnerabilities analyzed by the LLMs are extracted from the generated CoT. These labels are then compared with the actual labels of the code snippet in the dataset.

When the correct answer is generated, the reasoning chains underlying these steps are likely to be correct. Therefore, when the predicted label matches the true label, we accept the CoT. Since this is the basic principle that is upheld, we will only discuss in the following how to handle the cases that are not accepted, *i.e.*, when the predicted label does not match the true label.

In such cases where the judgment does not match, we combine the question with the LLMs' previous answer and append a statement such as "Please rethink again" at the end to encourage the LLMs to rethink and provide a new answer. If the predictions remain consistently inconsistent, the process will be repeated, and the number of times i is recorded until it reaches the limit number of repetitions N set by ourselves.

When the number of repetitions i exceeds the limit N, we cease the rethinking process. We utilize the correct label as known information and employ zero-shot learning to enable LLMs to generate a reasoning chain for smart contract vulnerability detection for that code snippet. Since the answers are known, it is challenging to automatically determine the correctness of the reasoning chain. Therefore, we allow the LLMs to check the answers themselves to discriminate the correctness of the reasoning chain. This process can also set the repetition limit N and check it multiple times as in the rethinking process. Upon exceeding the limit, we have the option to manually comment or deprecate the code snippet.

3.3. In-context learning stage

ICL is a paradigm that enables LLMs to leverage relevant domain knowledge without necessitating fine-tuning (Dong et al., 2022). This approach diminishes the necessity to gather extensive amounts of high-quality training data for new tasks, thereby mitigating the time and equipment costs associated with training.

The effectiveness of ICL hinges primarily on the quality, quantity, and order of demonstrations (Zhao et al., 2021). In Section 3.1, we select the top-k smart contract code snippets that are semantically most similar to the test code and generate the corresponding Code CoT for each in Section 3.2. In this stage, we combine each smart contract code snippet with its corresponding Code CoT to construct the demonstration. Subsequently, we combine the demonstration with the underlying prompts to form the prompts used for ICL.

An instance of the prompt template utilized by SmartGuard is presented in Fig. 4. In Underlying Prompt, we provide LLMs with a concise overview of the tasks and delineate the categories that require differentiation. Notice that Underlying Prompt can be tailored according to the specific focus of smart contract vulnerability detection. Moreover, to streamline data processing and facilitate critical information extraction, we impose restrictions on LLMs' responses. Specifically, we mandate the use of JSON format for answers and provide guidelines regarding the values to be filled in for each key. Additionally, we limit the word

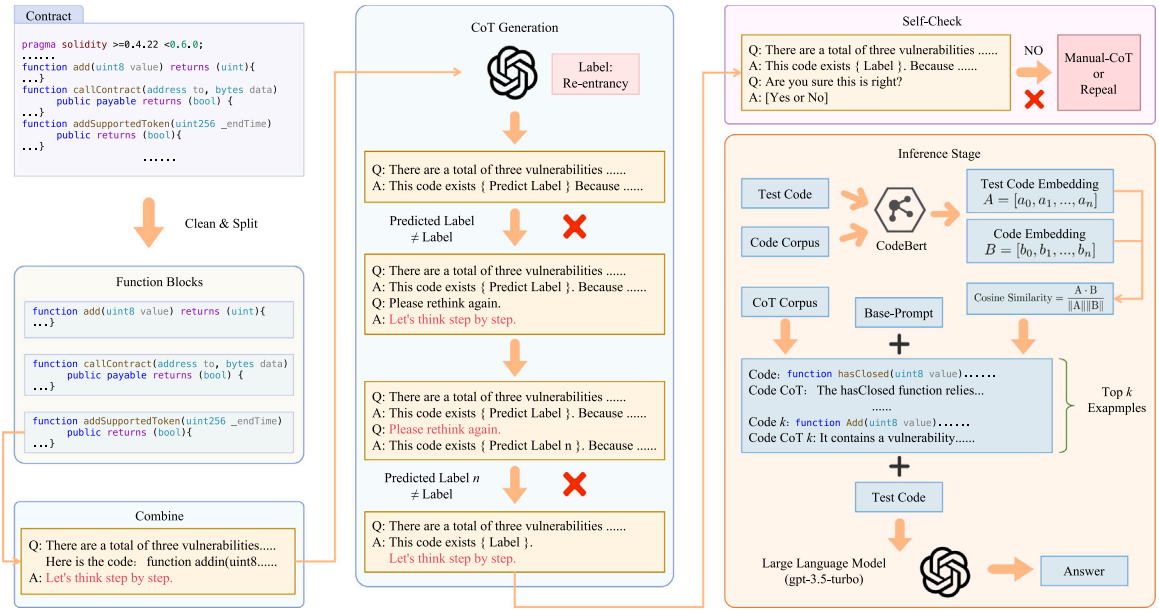


Fig. 3. Workflow of our self-check architecture.

count of reason to ensure the smart contract vulnerability detection report remains concise. For optimal performance in ICL, we include explicit prompts prior to the demonstrations. In Demonstrations, we assemble selected combinations of smart contract code and their corresponding CoT compositions. These combinations are arranged such that those with high similarity to the test code are positioned closer, while those with low similarity are placed farther away based on their semantic similarity to the test code. Furthermore, each combination is separated from others by Code k and Code CoT k to make it easier for LLMs to distinguish each demonstration. In Test Code, we combine the test code with the label "Test Code" and employ "Detect Vulnerability" to prompt LLMs to generate the test report.

As outlined in Section 2.4, CoTs are representations that include intermediate reasoning steps. In our SmartGuard system, these steps are structured as strings that specify the type of vulnerability and its resolution, as depicted in the Code CoT module in Fig. 4. When LLMs reason, they perform ICL based on the demonstrations provided in the input under our Underlying Prompt. This approach guides the LLMs to analyze both Code and Code CoT, enabling them to identify whether the code is vulnerable and to understand the reasons for the vulnerability. Subsequently, the LLMs apply this reasoning to evaluate the Test Code we provide.

3.4. Inference stage

As the scale of deep model learning has grown, the cost of both model training and model use has increased dramatically. In contrast, utilizing ChatGPT by way of API calls has emerged as the preferred choice for many individuals lacking the requisite hardware to accommodate the demands of large models.

Our proposed SmartGuard generates reports on smart contract vulnerabilities by interfacing with the API of GPT-3.5 Turbo, thereby obviating the need for training and deploying models. We submit the predefined prompt template (assembled in Section 3.3) to the GPT-3.5 Turbo API and extract crucial information upon receiving the response. In the event of format failure or access failure, we will resubmit the same request after a brief delay (1 s), repeating this process until the threshold for single-question fault tolerance is exceeded.

4. Metrics for performance evaluation

To evaluate the performance of our proposed framework SmartGuard, we employ four key metrics: accuracy, precision, recall, and F1-score.

Accuracy serves as a fundamental evaluation metric for measuring the ability of a classification model in making accurate predictions. It reflects the proportion of correct predictions (true positives and true negatives) made by the model out of all predictions.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{3}$$

Precision denotes the ratio of true positives to all examples predicted as positive by the model. It elucidates the accuracy of positive predictions, indicating the proportion of correctly identified positive instances among all instances labeled as positive by the model.

$$Precision = \frac{TP}{TP + FP} \tag{4}$$

Recall is employed to gauge the proportion of positive examples (true positives) accurately predicted by the model among all positive examples, including both true positives and false negatives. It indicates the sensitivity of the model in identifying positive instances, essentially measuring the rate at which positive examples are identified.

$$Recall = \frac{TP}{TP + FN} \tag{5}$$

F1 score serves as the harmonized average of precision and recall. High recall may decrease precision by increasing false positives, whereas high precision might lead to more false negatives. By combining precision and recall, F1 score offers a comprehensive performance metric. Hence, we prioritize the F1 score as the primary consideration for evaluating model performance.

$$F1 = \frac{2TP}{2TP + FN + FP} \tag{6}$$

5. Experiment settings

In this section, we present a detailed overview of our experiments, encompassing the sources of the smart contracts utilized, the composition of test set, the parameters of the framework employed, and the environmental conditions. We have provided public access to our code at https://github.com/KevinClaint/SmartGuard.

Underlying Prompt

There are a total of three vulnerabilities, namely Overflow-Underflow, Reentrancy,Timestamp-Dependency. Please help me determine if it contains one of the above vulnerabilities.
If yes, please use the JSON format as follows to tell me the answer, where the answer is either "yes" or "no".

```
{
    "answer" : "", "type" : "", "reason" : ""
}
```

If "yes", please specify in the type which type of vulnerability it belongs to (it can be more than one), and explain the reason in the reason field (not exceeding 50 words). If "no", please fill in "none" in both type and reason. Below I will provide you with n demo.

Demonstrations

Code 1:

```
function subAllocation(address sender) private {
  if (allocations[sender][0].releaseTime < block.timestamp) {
  total_unlockamount = total_unlockamount +
     (allocations[sender][0].balance);
      }
  else {
  total_lockamount = total_lockamount +
     (allocations[sender][1].balance);
}
```

Code CoT 1:

```
{
    "answer" : "yes",
    "type" : "Timestamp-Dependency",
    "reason" : "The subAllocation function uses block.timestamp for
    ↪ time-based conditions, which can be manipulated by miners to
    ↪ exploit time-dependent logic."
}
```

.....

Code k:

```
#Code k#
```

Code CoT k:

```
#Code CoT k#
```

Test Code

Test Code:

```
# Test Code #
```

Detect Vulnerability:

```
# Detection Report #
```

Fig. 4. Prompt template of SmartGuard.

5.1. Smart contract corpus

The smart contract data within our corpus originates from the dataset produced by Qian, Liu, Yin, and He (2023). They collected 514,880 functions from 42,910 smart contracts sourced from the Ether platform, GitHub repositories, and blog posts analyzing these contracts. This dataset involves four different vulnerabilities: re-entrancy, timestamp-dependency, integer overflow/underflow, and delegatecall.

Within our corpus, we exclusively utilize data pertinent to the labels of re-entrancy, timestamp-dependency, and integer overflow/underflow. After manual analysis, we find that there are some code snippets with different labels in this dataset, as depicted in Fig. 5. This is low quality for the discriminative deep learning model but is much higher quality data for the generative deep learning model we used (GPT-3.5 Turbo). This is attributed to the fact that accurately labeled diverse labels provide a more nuanced analysis of the code snippet.

Hence, we establish the association between smart contracts and labels within the dataset. The 898 smart contracts initially categorized with a single label are integrated into 662 smart contracts with multiple labels. Among these, 386 smart contracts are labeled as having no

Solidity Code

```
contract PIGGY_BANK {
    mapping (address => uint) public Accounts;
    function Collect(uint _am) public payable {
        if(msg.sender.call.value(_am)()) {
            Accounts[msg.sender] -= _am;
        }
    }
}
```

Fig. 5. An example of a smart contract with multiple vulnerabilities (re-entrancy, and integer overflow/underflow).

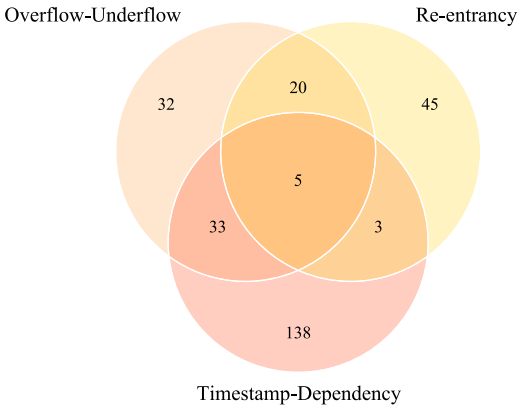


Fig. 6. Distribution of vulnerabilities in Smart Contract Corpus.

Table 1
The categories of test dataset.

None	Overflow-Underflow	Timestamp-Dependency	Re-Entrancy
2832	1333	1381	1343

vulnerability. The distribution of vulnerabilities among the remaining smart contracts is illustrated in Fig. 6.

5.2. Test dataset

We use the SolidiFI Benchmark dataset (Ghaleb & Pattabiraman, 2020) for our test dataset, which contains 9369 bugs across seven distinct types. This dataset is specifically designed for smart contract vulnerability detection, aligning closely with our research objectives. As one of the standard benchmark datasets in the field, it is widely utilized by researchers for comparison purposes. Utilizing this dataset allows us to benchmark our results against other studies, thereby reinforcing the scientific validity and credibility of our findings. Furthermore, since the bugs in this dataset are manually injected, it minimizes the occurrence of unlabeled bugs—a common issue in other datasets, including the one used for our corpus.

From this dataset, we specifically select 4057 bugs under the categories of re-entrancy, timestamp-dependency, and integer overflow/underflow to evaluate the performance of our framework, SmartGuard. Given that the SolidiFI Benchmark dataset is labeled based on smart contracts, we split the smart contracts within the dataset by functionality after removing annotations. We then label the code snippets containing bugs according to their original labels, while those devoid of bugs are labeled as bug-free. Subsequently, we compile the test dataset, comprising 7039 code snippets. The categories corresponding to each label are delineated in Table 1 below.

Table 2
Detailed hyper-parameter settings of SmartGuard.

Hyper-parameter	value
Maximum input length of code L	510
Rethinking limit N_t	6
Self-Check limit N_c	1

5.3. Framework parameters

In our experiments, the detailed Hyper-parameter settings within our framework are presented in Table 2. These values are determined based on the outcomes of our experiments and the methodological constraints employed.

LLMs possess the capability to handle an extensive number of tokens and can effectively process the vast majority of code. However, CodeBert, a Transformer-based deep learning model, can handle a maximum of 512 tokens. Additionally, the tokenizer adds special symbols to both the header and footer of the input code (each occupying one position) for CodeBert to recognize the code during processing. Consequently, the maximum permissible code length for generating semantic vectors using CodeBert is 510.

Given that excessive rethinks escalate both the economic and temporal costs of the experiment and diminish the quality of reasoning chain, we set the limitation of times N_t in the rethinking process to 6. This restriction ensures a reduced cost in generating reasoning chains while maintaining higher quality chains.

In this experiment, we set the limitation N_c of the self-check procedure to 1. If the answer to self-check is affirmative (Yes), we can reasonably infer that the LLMs deem the reasoning chain consistent with the result. Consequently, the result can be generated from this reasoning chain without the necessity of further reasoning. Conversely, if the response is negative (No), it is logical to assume that the LLMs perceive the reasoning chain as inconsistent with the result, necessitating the generation of a new reasoning chain for check or check again to avoid misjudgment. Since regenerating entails repeating the aforementioned process and is akin to re-checking, both of which could compromise the quality of the reasoning chain, it is imperative to minimize N_c. Therefore, N_c is set to 1 in this experiment.

5.4. Environments

All experiments are conducted using a computer equipped with an Intel Core i7-10700K @ 3.80 GHz (8-core) CPU and an NVIDIA GeForce RTX 3090 (24 GB) GPU. The operating system utilized is Windows 10.

6. Results and analysis

To ensure the generalizability of our framework, we selected three models—GPT-3.5 Turbo, ChatGLM3-6B, and Qwen-7B—as the LLMs on which our framework relies to conduct experiments on the test set. For each model, we use the self-check architecture mentioned in

Table 3
The results of different metrics for SmartGuard on LLMs.

Model	Approach	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
GPT-3.5 Turbo	SmartGuard+0-shot	89.10 ± 1.46	97.03 ± 0.28	83.65 ± 2.49	89.82 ± 1.51
	SmartGuard+2-shot	94.18 ± 1.03	94.85 ± 0.24	**95.06 ± 1.79**	**94.95 ± 0.93**
	SmartGuard+4-shot	**94.26 ± 0.86**	96.12 ± 0.22	93.83 ± 1.42	**94.95 ± 0.79**
	SmartGuard+8-shot	92.08 ± 1.47	96.74 ± 0.19	89.26 ± 2.64	92.83 ± 1.43
	SmartGuard+16-shot	91.32 ± 1.27	**97.11 ± 0.16**	87.54 ± 2.20	92.06 ± 1.23
ChatGLM3-6B	SmartGuard+0-shot	52.18 ± 0.90	63.91 ± 0.95	39.09 ± 1.53	48.50 ± 1.44
	SmartGuard+2-shot	66.21 ± 1.63	87.27 ± 0.73	48.41 ± 2.78	62.24 ± 2.48
	SmartGuard+4-shot	**72.34 ± 1.63**	**89.68 ± 0.41**	**58.75 ± 2.87**	**70.96 ± 2.18**
	SmartGuard+8-shot	58.78 ± 1.08	75.21 ± 0.72	42.45 ± 2.01	54.25 ± 1.82
	SmartGuard+16-shot	42.75 ± 0.92	50.71 ± 1.85	21.77 ± 1.70	30.45 ± 2.00
Qwen-7B	SmartGuard+0-shot	43.63 ± 0.00	52.19 ± 0.00	26.15 ± 0.00	34.84 ± 0.00
	SmartGuard+2-shot	84.47 ± 0.00	82.23 ± 0.00	**93.20 ± 0.00**	87.37 ± 0.00
	SmartGuard+4-shot	81.67 ± 0.00	82.15 ± 0.00	87.13 ± 0.00	84.57 ± 0.00
	SmartGuard+8-shot	**85.17 ± 0.00**	**83.41 ± 0.00**	92.70 ± 0.00	**87.81 ± 0.00**
	SmartGuard+16-shot	68.55 ± 0.00	77.71 ± 0.00	63.69 ± 0.00	70.01 ± 0.00

Section 3.2 to extend the CoT for selected kinds of smart contracts in Smart Contact Corpus, respectively. Afterward, we run our SmartGuard framework in the respective extended smart contract corpus as well as their CoT corpus. Based on the experimental results, we calculated four evaluation metrics: Accuracy, Precision, Recall, and F1-Score. To further ensure the accuracy and repeatability of the experiments, we conducted a total of 10 rounds of experiments. The results of these experiments are presented in Fig. 7 and Table 3. In addition, we present the various models integrated with our framework and the criteria used for identifying different vulnerabilities in Appendix A. The detailed results are displayed in Table A.5, Table A.6, and Table A.7. In terms of vulnerability detection, the Recall and F1-Score values are more indicative of the model's performance and overall effectiveness. Therefore, the following analysis primarily focuses on these two metrics.

From Table 3, we can clearly observe that SmartGuard equipped on GPT-3.5 Turbo achieves the highest recall value and F1-score value, which are 95.06% and 94.95%, respectively. Higher recall values indicate that our framework is able to accurately identify a significant portion of vulnerabilities. This reduces the probability of missed detection and helps mitigate the potential risk posed by vulnerable smart contracts to blockchain security. At the same time, a high F1-score ensures comprehensive performance and reduces the probability of false positives.

Fig. 7 illustrates the impact of the number of examples on LLMs' performance. All three LLMs start with 2 shots, demonstrating a leap in performance in vulnerability detection, *i.e.*, the LLMs efficiently learn the way of thinking related to vulnerability detection from the demonstrations. Simultaneously, we can also observe that the performance of the model has a significant degradation when it reaches 8 shots or more. We speculate that this is due to the input being too long, which diminishes the LLM's focus on task-relevant text.

In addition, Table 3 shows that the standard deviations of the three models across different indicators vary. Notably, the Qwen-7B model has a standard deviation of 0 for all indicators. This is because Qwen-7B exhibits exceptional stability, resulting in consistent outputs. Although the standard deviations were rounded to the second decimal point, the actual standard deviations were less than the minimum measurable value, indicating minimal variability rather than none at all. The standard deviations are calculated based on the results of 10 rounds of experiments, as shown in Table 3. For GPT-3.5 Turbo, the standard deviation trend aligns with the performance trend of SmartGuard, indicating consistent performance. However, for ChatGLM3-6B, the trend diverges. We speculate that while SmartGuard enhances performance on ChatGLM3-6B, it does not achieve the same level of effectiveness as on GPT-3.5 Turbo, resulting in greater variability in the generated results.

We acquired the results of smart contract vulnerability analysis for the SolidiFI Benchmark dataset from Ghaleb and Pattabiraman (2020) using various static analysis tools, including Oyente (Luu et al., 2016), Securify (Tsankov et al., 2018), Mythril (CONSENSYS, 2018), SmartCheck (Tikhomirov et al., 2018), Manticore (Mossberg et al., 2019), and Slither (Feist et al., 2019). These results are displayed in Table 4. In the table, "NA" denotes that the tool is unable to detect the corresponding vulnerability. Notice that the table indicates only the number of detected vulnerabilities, *i.e.*, the number of true positives. In Table 4, we observe that SmartGuard on GPT-3.5 Turbo performs optimally for the vulnerability Timestamp-dependency, while SmartGuard on Qwen-7B is optimal for both Overflow-Underflow and Re-entrancy. This phenomenon likely stems from differences in the inherent knowledge that various LLMs possess regarding specific vulnerabilities. Additionally, it is possible that different LLMs require tailored guidance for different vulnerabilities to enhance their performance further. This variability poses a challenge to our framework, as it highlights the need for customized strategies to maximize the effectiveness of each LLM for different types of vulnerabilities. Furthermore, most of the vulnerability detection tools, use Recall value as the main evaluation criterion. As we can see from Fig. 8, our framework, which is loaded on GPT-3.5 Turbo, achieves an optimal Recall value of 95.06%, outperforming all the tools listed. Among these tools, Manticore has the lowest value of 5.66% and Slither has the highest value of 53.90%. And our SmartGuard has an improvement of 41.16% compared to Slither. It also proves the superiority of our SmartGuard for vulnerability detection.

Table 4
Comparison of the number of true positives.

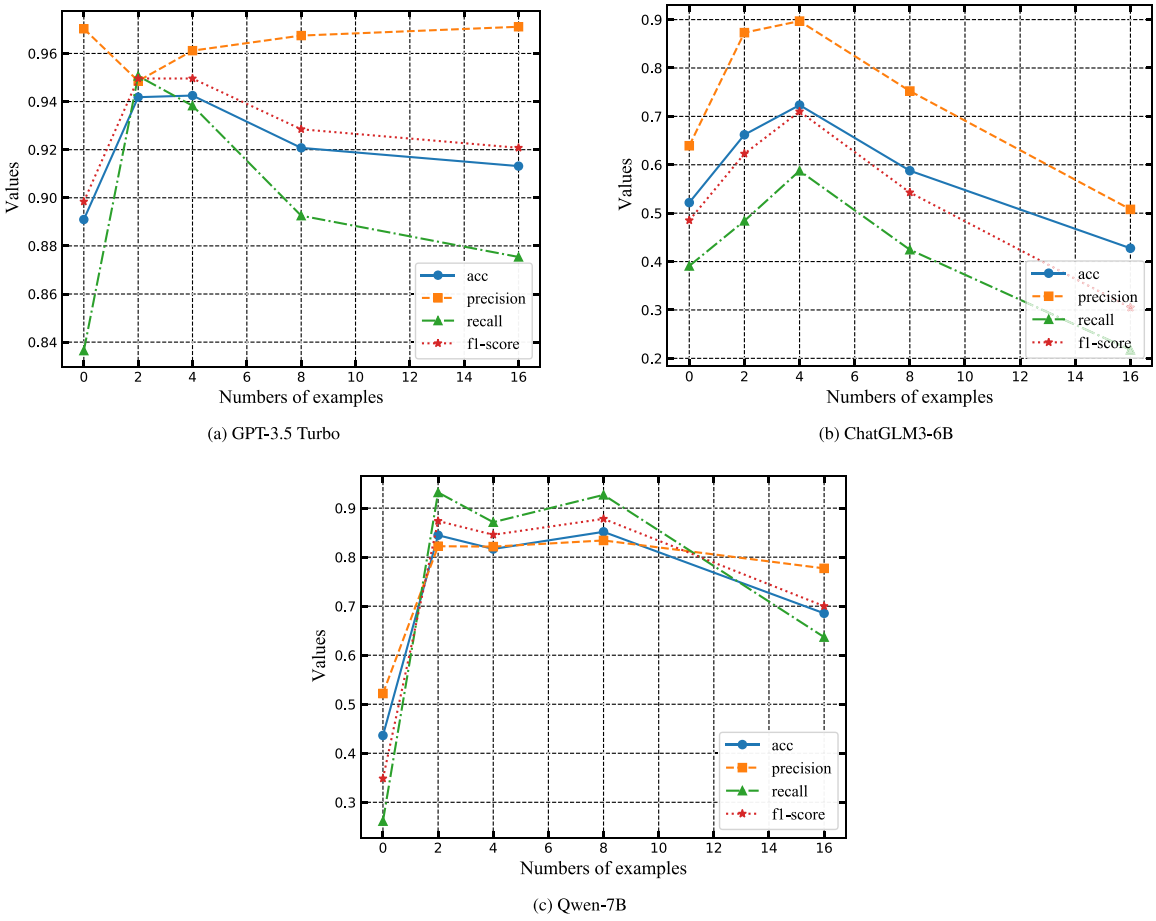
Tools	Overflow-Underflow	Timestamp-dependency	Re-entrancy
Inject Bugs	1333	1381	1343
Oyente (Luu et al., 2016)	435	0	335
Securify (Tsankov et al., 2018)	NA	NA	1111
Mythril (CONSENSYS, 2018)	264	571	258
SmartCheck (Tikhomirov et al., 2018)	261	479	0
Manticore (Mossberg et al., 2019)	137	NA	93
Slither (Feist et al., 2019)	NA	844	**1343**
GPT-3.5 Turbo+SG2	1305.2	**1285.1**	1266.4
ChatGLM3-6B+SG4	1228.7	478.4	676.5
Qwen-7B+SG2	**1333**	1105	**1343**

7. Discussion

This section will delve into the strengths of our work, acknowledge the limitations, and suggest directions for its application.

7.1. Advantages

A distinctive feature of SmartGuard is its ability to simultaneously detect multiple vulnerabilities within a single codebase, setting it apart



(a) GPT-3.5 Turbo

(b) ChatGLM3-6B

(c) Qwen-7B

Fig. 7. Results of our SmartGuard on three LLMs.

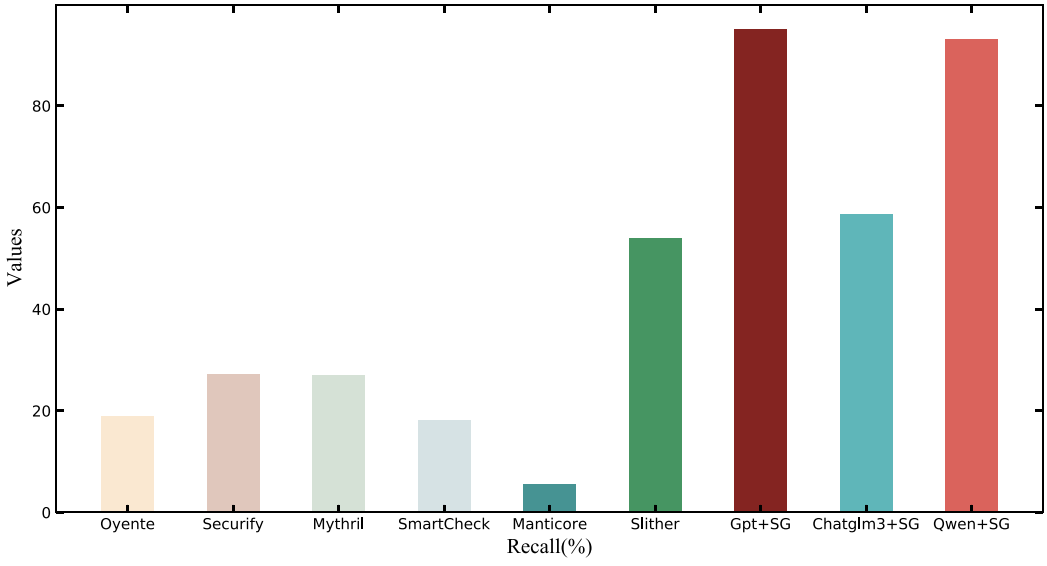


Fig. 8. Comparison of Recall Rates of Various Tools.

from other deep learning models, which are typically discriminative. However, the availability of high-quality datasets containing multiple labels for a single data point is limited, which is essential for a more comprehensive assessment of SmartGuard's capabilities. We anticipate that the development of such datasets in the future will be crucial for enhancing the evaluation process of our framework.

Furthermore, our framework can be extended to detect both common and self-defined vulnerabilities, which is another advantage that distinguishes it from conventional vulnerability detection tools. Most of these traditional tools rely on predefined rules. The following steps outline how to extend our framework: First, gather a dataset of relevant vulnerabilities. Next, preprocess the dataset by performing tasks such

Table A.5
The results of metrics for each vulnerability type of GPT-3.5 Turbo.

Vulnerability	Approach	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
Re-Entrancy	SmartGuard+0-shot	92.60 ± 1.66	98.43 ± 0.27	88.54 ± 2.82	93.22 ± 1.59
	SmartGuard+2-shot	**94.86 ± 1.36**	96.68 ± 0.37	**94.30 ± 2.37**	**95.47 ± 1.25**
	SmartGuard+4-shot	92.46 ± 1.97	97.10 ± 0.37	89.55 ± 3.35	93.17 ± 1.91
	SmartGuard+8-shot	89.61 ± 2.07	98.42 ± 0.23	83.25 ± 3.65	90.20 ± 2.18
	SmartGuard+16-shot	93.99 ± 1.99	**98.95 ± 0.19**	90.51 ± 3.49	94.54 ± 1.90
Timestamp-Dependency	SmartGuard+0-shot	90.56 ± 3.45	**98.05 ± 0.31**	85.47 ± 5.83	91.33 ± 3.42
	SmartGuard+2-shot	94.28 ± 1.54	96.98 ± 0.30	93.06 ± 2.59	94.98 ± 1.41
	SmartGuard+4-shot	**94.77 ± 2.22**	97.53 ± 0.33	**93.37 ± 3.68**	**95.41 ± 2.05**
	SmartGuard+8-shot	92.64 ± 2.95	97.82 ± 0.32	89.34 ± 5.00	93.39 ± 2.82
	SmartGuard+16-shot	86.57 ± 2.70	97.89 ± 0.16	78.60 ± 4.76	87.19 ± 2.89
Integer Overflow/Underflow	SmartGuard+0-shot	84.08 ± 4.41	94.33 ± 0.53	76.82 ± 7.90	84.68 ± 4.92
	SmartGuard+2-shot	93.40 ± 1.71	91.22 ± 0.59	97.91 ± 2.88	94.45 ± 1.53
	SmartGuard+4-shot	**95.54 ± 1.08**	93.91 ± 0.38	**98.62 ± 1.96**	**96.21 ± 0.97**
	SmartGuard+8-shot	93.98 ± 1.44	94.32 ± 0.45	95.21 ± 2.36	94.77 ± 1.32
	SmartGuard+16-shot	93.47 ± 2.31	**94.74 ± 0.33**	93.81 ± 4.05	94.27 ± 2.16

Table A.6
The results of metrics for each vulnerability type of ChatGLM3-6B.

Vulnerability	Approach	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
Re-Entrancy	SmartGuard+0-shot	64.49 ± 2.53	93.74 ± 1.02	40.95 ± 4.44	57.00 ± 4.37
	SmartGuard+2-shot	60.76 ± 2.40	**99.42 ± 0.29**	31.91 ± 4.15	48.31 ± 4.81
	SmartGuard+4-shot	**71.03 ± 1.91**	98.47 ± 0.36	**50.37 ± 3.43**	**66.65 ± 3.03**
	SmartGuard+8-shot	58.55 ± 2.38	79.58 ± 2.48	37.48 ± 3.79	50.96 ± 3.99
	SmartGuard+16-shot	35.62 ± 1.74	27.26 ± 8.26	7.21 ± 2.64	11.40 ± 4.02
Timestamp-Dependency	SmartGuard+0-shot	50.83 ± 0.27	87.20 ± 1.96	18.10 ± 0.00	29.98 ± 0.12
	SmartGuard+2-shot	54.03 ± 0.89	**99.45 ± 0.44**	21.06 ± 1.52	34.76 ± 2.08
	SmartGuard+4-shot	**61.88 ± 1.82**	99.42 ± 0.28	**34.64 ± 3.16**	**51.38 ± 3.45**
	SmartGuard+8-shot	52.54 ± 2.11	75.94 ± 3.28	26.90 ± 3.20	39.73 ± 3.90
	SmartGuard+16-shot	46.02 ± 2.77	57.95 ± 4.54	26.10 ± 4.25	35.99 ± 4.88
Integer Overflow/Underflow	SmartGuard+0-shot	41.20 ± 0.38	48.90 ± 0.27	58.96 ± 0.00	53.47 ± 0.16
	SmartGuard+2-shot	84.11 ± 2.70	81.56 ± 0.60	**93.37 ± 5.15**	**87.07 ± 2.60**
	SmartGuard+4-shot	**84.32 ± 3.43**	**82.50 ± 1.12**	92.18 ± 5.90	**87.07 ± 3.30**
	SmartGuard+8-shot	65.38 ± 2.47	72.59 ± 1.46	63.56 ± 4.28	67.77 ± 3.03
	SmartGuard+16-shot	46.58 ± 1.46	55.90 ± 1.98	31.96 ± 3.02	40.67 ± 2.93

as segmentation and removal of irrelevant information. The processed data is then converted into semantic vectors using CodeBERT, enabling the framework to retrieve code snippets semantically similar to the test snippets. Subsequently, modify the prompt template. For common vulnerabilities, simply update the official name and the total number of vulnerabilities. For uncommon or self-defined vulnerabilities, provide a description of their logical rules. This will assist the LLMs in understanding the logical structure required for detection and leveraging relevant domain knowledge for analysis. Finally, adjust the hyperparameters to optimize the framework's performance. This step reduces financial costs by minimizing unnecessary LLM queries and enhances the quality of the reasoning chain.

7.2. Limitations

While the majority of the code is logically structured in a way that closely aligns with its spatial structure, there are instances where critical information is lost due to limitations on input code length. Currently, our approach faces challenges in effectively addressing such cases. Additionally, we recognize the need for a more comprehensive evaluation of our framework's performance, extending beyond datasets where each data point corresponds to a single label. A key distinction of our framework, compared to other discriminative deep learning models, is its ability to detect multiple vulnerabilities in a single codebase. However, high-quality datasets with multiple labels for a single data point are still lacking. We anticipate that the emergence of such datasets will enable more robust evaluations of our framework in the future. Furthermore, while the use of LLMs tends to incur higher financial and time costs, we believe these costs are negligible in comparison to the potential losses caused by vulnerabilities in smart contracts. With the rapid advancement of LLMs, we expect that these costs will decrease significantly in the future.

7.3. Applications

Our framework has broad applicability in fields involving blockchain smart contracts, such as finance and healthcare. By detecting and repairing security vulnerabilities in smart contracts, it reduces the risk of hacking and safeguards user assets. Additionally, it can provide contract risk assessment services for enterprises and developers, helping them understand contract security and make necessary improvements. Furthermore, our framework can be integrated into the software development life cycle to continuously monitor smart contract code, thereby improving development efficiency.

8. Conclusion and future work

This paper introduces SmartGuard, an innovative framework designed to enhance the detection of smart contract vulnerabilities by integrating LLMs. Our approach leverages the semantic analysis capabilities of LLMs to retrieve and analyze code snippets, generates CoTs prompts to assist in identifying vulnerabilities, and utilizes these generated CoTs to improve vulnerability detection. Additionally, we propose a self-checking architecture that maximizes the use of small amounts of labeled data to generate high-quality inference chains, leveraging the self-correcting capabilities of LLMs. The experimental results demonstrate that our framework achieves a recall of 95.06% and an F1-score of 94.95% on the test set, indicating strong performance and significant potential for vulnerability detection. Furthermore, we

Table A.7
The results of metrics for each vulnerability type of Qwen-7B.

Vulnerability	Approach	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
Re-Entrancy	SmartGuard+0-shot	53.58 ± 0.52	72.11 ± 1.49	31.35 ± 0.00	43.70 ± 0.27
	SmartGuard+2-shot	**94.72 ± 0.05**	**91.59 ± 0.07**	**100.00 ± 0.00**	**95.61 ± 0.04**
	SmartGuard+4-shot	94.57 ± 0.17	91.37 ± 0.25	**100.00 ± 0.00**	95.49 ± 0.14
	SmartGuard+8-shot	92.07 ± 0.24	89.64 ± 0.35	97.47 ± 0.00	93.39 ± 0.19
	SmartGuard+16-shot	65.75 ± 0.37	78.33 ± 0.71	55.85 ± 0.00	65.20 ± 0.24
Timestamp-Dependency	SmartGuard+0-shot	58.76 ± 0.42	78.02 ± 1.07	40.48 ± 0.00	53.30 ± 0.25
	SmartGuard+2-shot	84.58 ± 0.06	92.45 ± 0.12	80.01 ± 0.00	85.79 ± 0.05
	SmartGuard+4-shot	61.88 ± 1.82	**99.42 ± 0.28**	34.64 ± 3.16	51.38 ± 3.45
	SmartGuard+8-shot	**85.77 ± 0.29**	93.64 ± 0.53	**81.03 ± 0.00**	**86.88 ± 0.23**
	SmartGuard+16-shot	81.29 ± 0.39	87.67 ± 0.65	78.93 ± 0.00	83.07 ± 0.29
Integer Overflow/Underflow	SmartGuard+0-shot	18.19 ± 0.50	11.06 ± 0.17	6.08 ± 0.00	7.84 ± 0.04
	SmartGuard+2-shot	74.07 ± 0.05	68.84 ± 0.04	**100.00 ± 0.00**	81.54 ± 0.03
	SmartGuard+4-shot	74.87 ± 0.13	69.51 ± 0.11	**100.00 ± 0.00**	82.01 ± 0.07
	SmartGuard+8-shot	**77.62 ± 0.13**	**71.91 ± 0.12**	**100.00 ± 0.00**	**83.66 ± 0.08**
	SmartGuard+16-shot	58.35 ± 0.14	66.18 ± 0.19	55.81 ± 0.00	60.56 ± 0.08

conducted experiments on multiple models to evaluate the generalizability of our framework. We also examined the impact of the number of exemplars on experimental results and found that it significantly influences outcomes. Moreover, the optimal number of exemplars is not fixed; it depends on factors such as the performance and technology of the LLM used by SmartGuard, warranting further investigation.

Our framework has certain limitations, including higher time and economic costs associated with the use of LLMs, as well as constraints on input length. Additionally, we are working to identify test datasets where multiple vulnerabilities within a single example are accurately labeled, enabling a more precise evaluation of our framework's performance. These efforts aim to further strengthen SmartGuard's ability to improve smart contract security.

In the future, we plan to enhance SmartGuard's performance in detecting true positives by refining its framework design. Furthermore, we aim to identify more suitable test datasets to accurately assess its performance. These initiatives will contribute to advancing SmartGuard's capabilities in enhancing the security of smart contracts.

CRediT authorship contribution statement

Hao Ding: Writing – review & editing, Software, Methodology, Visualization, Formal analysis. **Yizhou Liu:** Writing–original draft, Software, Methodology, Data curation, Validation. **Xuefeng Piao:** Supervision, Writing – review & editing, Conceptualization, Investigation, Funding acquisition. **Huihui Song:** Supervision, Resources, Conceptualization. **Zhenzhou Ji:** Supervision, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was funded by the National Natural Science Foundation of China, grant number U23B2051 and the Natural Science Foundation of Shandong Province, China, grant number ZR2023ME162.

Appendix A. Detail

See Tables A.5–A.7.

Data availability

Data will be made available on request.

References

Alharby, M., & Van Moorsel, A. (2017). Blockchain-based smart contracts: A systematic mapping study. arXiv preprint arXiv:1710.06372.

Bai, X., Cheng, Z., Duan, Z., & Hu, K. (2018). Formal modeling and verification of smart contracts. In *Proceedings of the 2018 7th international conference on software and computer applications* (pp. 322–326).

Boi, B., Esposito, C., & Lee, S. (2024). Smart contract vulnerability detection: The role of large language model (llm). *ACM SIGAPP Applied Computing Review*, *24*, 19–29.

Brent, L., Jurisevic, A., Kong, M., Liu, E., Gauthier, F., Gramoli, V., et al. (2018). Vandal: A scalable security analysis framework for smart contracts. arXiv preprint arXiv:1809.03981.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, *33*, 1877–1901.

Chen, C., Su, J., Chen, J., Wang, Y., Bi, T., Yu, J., et al. (2023). When chatgpt meets smart contract vulnerability detection: How far are we? *ACM Transactions on Software Engineering and Methodology*.

Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., et al. (2023). Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, *24*, 1–113.

Clack, C. D., Bakshi, V. A., & Braine, L. (2016). Smart contract templates: foundations, design landscape and research directions. arXiv preprint arXiv:1608.00771.

CONSENSYS (2018). Mythril. https://github.com/Consensys/mythril.

Dong, Q., Li, L., Dai, D., Zheng, C., Wu, Z., Chang, B., et al. (2022). A survey on in-context learning. arXiv preprint arXiv:2301.00234.

Du, Z., Qian, Y., Liu, X., Ding, M., Qiu, J., Yang, Z., et al. (2022). Glm: General language model pretraining with autoregressive blank infilling. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 320–335).

E., Shishkin (2019). Debugging smart contract's business logic using symbolic model checking. *Programming and Computer Software*, *45*, 590–599.

Feist, J., Grieco, G., & Groce, A. (2019). Slither: a static analysis framework for smart contracts. In *2019 IEEE/ACM 2nd international workshop on emerging trends in software engineering for blockchain* (pp. 8–15). IEEE.

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., et al. (2020). Codebert: A pre-trained model for programming and natural languages. arXiv preprint arXiv:2002.08155.

Ghaleb, A., & Pattabiraman, K. (2020). How effective are smart contract analysis tools? evaluating smart contract static analysis tools using bug injection. In *Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis*.

Gogineni, A. K., Swayamjyoti, S., Sahoo, D., Sahu, K. K., & Kishore, R. (2020). Multi-class classification of vulnerabilities in smart contracts using awd-lstm, with pre-trained encoder inspired from natural language processing. *IOP SciNotes*, *1*, Article 035002.

Gupta, N. A., Bansal, M., Sharma, S., Mehrotra, D., & Kakkar, M. (2024). Detection of vulnerabilities in blockchain smart contracts using deep learning. *Wireless Networks*, 1–17.

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., et al. (2023). . Large language models for software engineering: A systematic literature review. arXiv preprint arXiv:2308.10620.

Huang, J., Zhou, K., Xiong, A., & Li, D. (2022). Smart contract vulnerability detection model based on multi-task learning. *Sensors*, *22*(1829).

Jiang, B., Liu, Y., & Chan, W. K. (2018). Contractfuzzer: Fuzzing smart contracts for vulnerability detection. In *Proceedings of the 33rd ACM/IEEE international conference on automated software engineering* (pp. 259–269).

Kalra, S., Goel, S., Dhawan, M., & Sharma, S. (2018). Zeus: analyzing safety of smart contracts. In *Ndss* (pp. 1–12).

Kang, S., Yoon, J., & Yoo, S. (2023). Large language models are few-shot testers: Exploring llm-based general bug reproduction. In *2023 IEEE/ACM 45th international conference on software engineering* (pp. 2312–2323). IEEE.

Kim, G., Baldi, P., & McAleer, S. (2024). Language models can solve computer tasks. *Advances in Neural Information Processing Systems*, *36*.

Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., & Iwasawa, Y. (2022). Large language models are zero-shot reasoners. *Advances in Neural Information Processing Systems*, *35*, 22199–22213.

Levy, I., Bogin, B., & Berant, J. (2022). Diverse demonstrations improve in-context compositional generalization. arXiv preprint arXiv:2212.06800.

Li, X., Jiang, P., Chen, T., Luo, X., & Wen, Q. (2020). A survey on the security of blockchain systems. *Future Generation Computer Systems*, *107*, 841–853.

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., et al. (2019). Roberta: A robustly optimized bert pretraining approach. arXiv preprint arXiv:1907.11692.

Liu, J., Shen, D., Zhang, Y., Dolan, B., Carin, L., & Chen, W. (2021). What makes good in-context examples for gpt-3?. arXiv preprint arXiv:2101.06804.

Lu, Y. (2019). The blockchain: State-of-the-art and research challenges. *Journal of Industrial Information Integration*, *15*, 80–90.

Lu, Y., Bartolo, M., Moore, A., Riedel, S., & Stenetorp, P. (2022). Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. In S. Muresan, P. Nakov, & A. Villavicencio (Eds.), *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: long papers)* (pp. 8086–8098). Dublin, Ireland: Association for Computational Linguistics.

Lutz, O., Chen, H., Fereidooni, H., Sendner, C., Dmitrienko, A., Sadeghi, A. R., et al. (2021). Escort: ethereum smart contracts vulnerability detection using deep neural network and transfer learning. arXiv preprint arXiv:2103.12607.

Luu, L., Chu, D.-H., Olickel, H., Saxena, P., & Hobor, A. (2016). Making smart contracts smarter. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security* (pp. 254–269).

Mossberg, M., Manzano, F., Hennenfent, E., Groce, A., Grieco, G., Feist, J., et al. (2019). Manticore: A user-friendly symbolic execution framework for binaries and smart contracts. In *2019 34th IEEE/ACM international conference on automated software engineering* (pp. 1186–1189).

Park, D., Zhang, Y., Saxena, M., Daian, P., & Roşu, G. (2018). A formal verification tool for ethereum vm bytecode. In *Proceedings of the 2018 26th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering* (pp. 912–915).

Qian, P., Liu, Z., Yin, Y., & He, Q. (2023). Cross-modality mutual learning for enhancing smart contract vulnerability detection on bytecode. In *Proceedings of the ACM web conference 2023 WWW '23* (pp. 2220–2229). New York, NY, USA: Association for Computing Machinery.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. (2019). Language models are unsupervised multitask learners. *OpenAI Blog*, *1*(9).

Saunders, W., Yeh, C., Wu, J., Bills, S., Ouyang, L., Ward, J., et al. (2022). Self-critiquing models for assisting human evaluators. arXiv preprint arXiv:2206.05802.

Sayeed, S., Marco-Gisbert, H., & Caira, T. (2020). Smart contract: Attacks and protections. *Ieee Access*, *8*, 24416–24427.

Shum, K., Diao, S., & Zhang, T. (2023). Automatic prompt augmentation and selection with chain-of-thought from labeled data. arXiv preprint arXiv:2302.12822.

Sun, Y., Wu, D., Xue, Y., Liu, H., Ma, W., Zhang, L., et al. (2024). Llm4vuln: A unified evaluation framework for decoupling and enhancing llms' vulnerability reasoning. arXiv preprint arXiv:2401.16185.

Sun, T., & Yu, W. (2020). A formal verification framework for security issues of blockchain smart contracts. *Electronics*, *9*(255).

Tann, W. J.-W., Han, X. J., Gupta, S. S., & Ong, Y.-S. (2018). Towards safer smart contracts: A sequence learning approach to detecting security threats. arXiv preprint arXiv:1811.06632.

Thoppilan, R., Freitas, D. De., Hall, J., Shazeer, N., Kulshreshtha, A., Cheng, H.-T., et al. (2022). Lamda: Language models for dialog applications. arXiv preprint arXiv:2201.08239.

Tikhomirov, S., Voskresenskaya, E., Ivanitskiy, I., Takhaviev, R., Marchenko, E., & Alexandrov, Y. (2018). Smartcheck: Static analysis of ethereum smart contracts. In *Proceedings of the 1st international workshop on emerging trends in software engineering for blockchain* (pp. 9–16).

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., et al. (2023). Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971.

Tsankov, P., Dan, A., Drachsler-Cohen, D., Gervais, A., Buenzli, F., & Vechev, M. (2018). Securify: Practical security analysis of smart contracts. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security* (pp. 67–82).

Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., et al. (2022). Emergent abilities of large language models. arXiv preprint arXiv:2206.07682.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, *35*, 24824–24837.

Xu, M., Guo, Y., Liu, C., Hu, Q., Yu, D., Xiong, Z., et al. (2024). Exploring blockchain technology through a modular lens: A survey. *ACM Computing Surveys*, *56*, 1–39.

Yang, J., Jin, H., Tang, R., Han, X., Feng, Q., Jiang, H., et al. (2024). Harnessing the power of llms in practice: A survey on chatgpt and beyond. *ACM Transactions on Knowledge Discovery from Data*, *18*, 1–32.

Zeng, A., Liu, X., Du, Z., Wang, Z., Lai, H., Ding, M., et al. (2022). Glm-130b: An open bilingual pre-trained model. arXiv preprint arXiv:2210.02414.

Zhang, Z., Zhang, A., Li, M., & Smola, A. (2022). Automatic chain of thought prompting in large language models. arXiv preprint arXiv:2210.03493.

Zhao, Z., Wallace, E., Feng, S., Klein, D., & Singh, S. (2021). Calibrate before use: Improving few-shot performance of language models. In M. Meila, & T. Zhang (Eds.), *Proceedings of machine learning research: vol. 139*, *Proceedings of the 38th international conference on machine learning* (pp. 12697–12706). PMLR.